ಬಿ.ಎಂ.ಎಸ್. ತಾಂತ್ರಿಕ ಮತ್ತು ವ್ಯವಸ್ಥಾಪನಾ ಮಹಾವಿದ್ಯಾಲಯ
BMS INSTITUTE OF TECHNOLOGY & MANAGEMENT
(Autonomous Institution Under VTU)
Yelahanka, Bengaluru -560064

A Course Project Work Report

on

"ENTERPRISE SERVICE DESK & INCIDENT MANAGEMENT PLATFORM"

Submitted in partial fulfillment of the requirements of CCA Phase-2 for

BUILDING ENTERPRISE APPLICATIONS (BCS703B)

in

INFORMATION SCIENCE AND ENGINEERING

By

B. VEENA **USN: 1BY23IS047**

C. RACHANA **USN: 1BY23IS055**

Under the guidance of

Mrs. Malini M
Assistant Professor

2026-2027

ABSTRACT

This Phase-2 work turns the Phase-1 design of an Enterprise Service Desk & Incident Management Platform into a working full-stack application for the IT service-management domain. The implemented three-tier architecture uses a React 18 and Bootstrap 5 single-page client served by Nginx, a Spring Boot REST API secured with Spring Security, JSON Web Tokens and BCrypt with three roles (employee, support engineer, administrator), and Spring Data JPA/Hibernate persistence on MySQL 8.4, packaged as a Docker Compose stack. The repository separates frontend, controller, service, repository, model, DTO and security packages, with tests, deployment files, scripts and documentation in their own folders; the default backend runs on Spring Boot 3.5.16 and an identical hardened module runs on Spring Boot 4.1.1. Static analysis with SpotBugs 4.9.8 reported zero bugs, and therefore zero critical bugs, in the recorded build of 16 October 2026; Checkstyle (Google Java Style) warnings were reduced from 116 to 0 in both backend modules, and npm audit reports no known vulnerabilities after the frontend build toolchain was upgraded. Testing comprises two JUnit 5 unit tests of the dashboard metrics and six Spring MockMvc API tests covering input validation, authentication, authorization and the incident endpoints; all eight passed in the recorded run. A 20-step end-to-end smoke test and a single verification script were added to re-run the tests, analysers and image scans and to exercise the complete login, create, assign, resolve and report workflow against MySQL on the final source. The operational outcome is a role-based incident workflow with audit history, in-app notifications and administrator reporting, with the review findings on secrets, demo access, schema safety and report accuracy corrected in the source.

REPOSITORY SETUP & CODE STRUCTURE

1.1 Repository Details

- **Repository Link: [insert the public GitHub URL here after pushing, e.g. https://github.com/<account>/enterprise-service-desk]**

  The Git repository is prepared locally with 89 tracked files on the main and dev branches (Figure 1.1). It has not been pushed yet, so no public URL is claimed. Publish it with:

```
git remote add origin https://github.com/<account>/enterprise-service-desk.git
git push -u origin main
git push -u origin dev
```

- **Branching Practice:** main holds stable, verified code; dev is the integration branch; each module or fix is developed on a feature/<name> branch and merged into dev through a pull request; dev is merged into main only after scripts/verify-all passes. .env files, database backups and tokens are never committed.

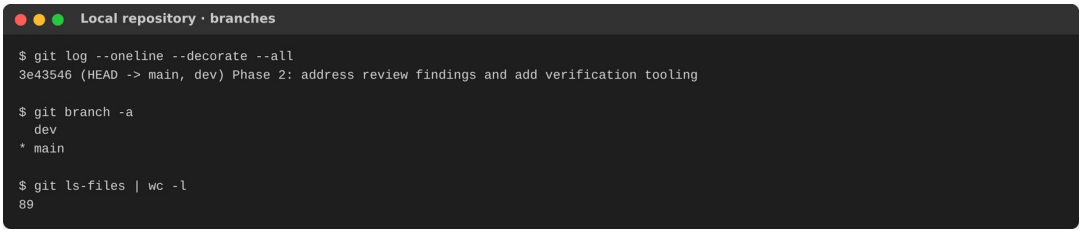


Figure 1.1: Local repository with main and dev branches

1.2 Technology Environment

Layer	Technology used
Presentation	React 18.3.1, Bootstrap 5.3.8, Vite 7.3.7 (build tool)
Web server / proxy	Nginx (alpine): serves the built SPA and forwards /api to the backend
REST API	Java 17; Spring Boot 3.5.16 (default backend) and 4.1.1 (hardened backend)
Security	Spring Security (stateless), JJWT 0.12.6 HMAC-SHA JWT valid 24 h, BCrypt, role and ownership checks
Persistence	Spring Data JPA, Hibernate ORM, MySQL 8.4 (H2 in MySQL mode for tests)
Testing	JUnit 5, Spring Boot Test + MockMvc, Maven Surefire; Node.js 22 end-to-end smoke test
Code quality	SpotBugs 4.9.8, Checkstyle 9.3 (Google Java Style), npm audit, Trivy image scan
Deployment	Docker Compose; multi-stage Dockerfiles; distroless and Chainguard JRE runtime images

1.3 Project Directory Structure

```
service-desk-phase2/
├── frontend/                    # React UI: src/main.jsx, src/style.css, package.json
├── backend/                     # Spring Boot 3.5 API (default deployment)
│   ├── src/main/java/com/bea/servicedesk/
│   │   ├── controller/          # REST endpoints (ApiController)
│   │   ├── service/             # Workflow, access rules, metrics
│   │   ├── repo/                # Spring Data JPA repositories
│   │   ├── model/               # JPA entities and enums
│   │   ├── dto/                 # Validated request records
│   │   └── config/              # Security, JWT, optional demo seed
│   ├── src/test/java/...        # JUnit unit + MockMvc API tests
│   ├── pom.xml, Dockerfile
├── backend-spring4-test/        # Same code on Spring Boot 4.1.1 (hardened)
├── scripts/                     # verify-all, smoke-test, backup-db, pin-images
├── docs/                        # Operations, test cases, review response
├── evidence/                    # Archived and fresh analysis/test output
├── compose.yaml                 # MySQL + backend + frontend
├── compose.secure.yaml          # Override: Spring Boot 4 backend, schema locked
├── Dockerfile, nginx.conf       # Frontend image and /api reverse proxy
├── .env.example                 # Required variables, no values
└── README.md                    # Setup instructions and run commands
```

1.4 Execution & Setup Commands

With Docker (recommended):

```
cp .env.example .env            # set DB_PASSWORD, MYSQL_ROOT_PASSWORD, JWT_SECRET
docker compose up --build -d     # open http://localhost:8088
# optional Spring Boot 4 backend (back up the database first):
scripts/backup-db.sh
docker compose -f compose.yaml -f compose.secure.yaml up --build -d
```

Without Docker (Java 17, Maven 3.9, Node 20.19+, MySQL 8):

```
cd backend && mvn spring-boot:run      # needs DB_URL, DB_USER, DB_PASSWORD, JWT_SECRET
cd frontend && npm ci && npm run dev   # open http://localhost:5173
cd backend && mvn clean test            # unit + API tests
scripts/verify-all.ps1  (or .sh)        # all tests, analysis, scans, end-to-end check
```

STATIC CODE ANALYSIS

- **Tools Used:** SpotBugs 4.9.8 (bytecode bug-pattern analysis); Checkstyle 9.3 with google_checks.xml, the version used by maven-checkstyle-plugin 3.5.0; npm audit for frontend dependencies; Trivy for container images (run by scripts/verify-all).
- **Quality Metrics Summary:**
  - Critical Bugs: **0** (SpotBugs total_bugs = 0)
  - Vulnerabilities: **0** (npm audit, frontend dependencies)
  - Code Warnings: **0** Checkstyle warnings after refactoring (116 before)

Metric	Result	Source and date
Bugs (all ranks, incl. critical)	0 bug instances; 7 packages, 32 classes analysed	SpotBugs XML, Spring Boot 4 build of 16 Oct 2026 (before the 18 Oct refactoring); verify-all reruns it
Dependency vulnerabilities	0 (previously 1 high, 1 moderate: esbuild/Vite 5.4)	npm audit, 18 Oct 2026
Style warnings	116 → 0 in backend/ and backend-spring4-test/	Checkstyle 9.3, 18 Oct 2026 (116 also in the 16 Oct XML)
Container image CVEs (HIGH/CRITICAL)	[fill in from verify-all trivy-*.log]	Trivy on rebuilt images; not run yet

- **Code Issues Resolved:**

Detected	File Name	Fix / Refactoring Applied
AvoidStarImport (21): wildcard imports such as model.* and repo.*	ApiController.java, IncidentService.java, model/*.java	Replaced with explicit imports in the required order
NeedBraces (20): if/for bodies without braces	ApiController.java, AccessService.java, IncidentService.java	Added braces; split IncidentService.update() into assign() and changeStatus() without behaviour change
MissingJavadocType/Method (52)	All public classes, records and methods	Added Javadoc describing purpose, parameters and access rules
Indentation (16): 4-space blocks	config/JwtService.java	Reformatted to Google style; extracted constants
npm audit: esbuild ≤ 0.24.2 via Vite 5.4 (GHSA-67mh-4wv8-2f99)	frontend/package.json	Upgraded to Vite 7.3.7, exact versions, package-lock.json, npm ci in the image build
Manual review: Spring Boot 4-only imports in Spring Boot 3 tests (compile failure)	backend/src/test/.../ApiIntegrationTest.java	Used the Spring Boot 3 Jackson and MockMvc packages
Manual review: 400/401 turned into 403 when forwarded to /error	config/SecurityConfig.java	Permitted /error and enabled RFC 9457 problem details

Verification Proof:

Checkstyle 9.3 · Google Java Style · 18 Oct 2026

```
# BEFORE: original uploaded source (backend-spring4-test)
$ java -jar checkstyle-9.3-all.jar -c /google_checks.xml src/main/java
Starting audit...
[WARN] model/Priority.java:3:1: Missing a Javadoc comment. [MissingJavadocType]
[WARN] model/Role.java:3:1: Missing a Javadoc comment. [MissingJavadocType]
[WARN] model/Notification.java:3:27: Using the '.*' form of import should be avoided - jakarta.persistence.*. [AvoidStarImport]
[WARN] model/Notification.java:6:1: Missing a Javadoc comment. [MissingJavadocType]
[WARN] model/Incident.java:3:27: Using the '.*' form of import should be avoided - jakarta.persistence.*. [AvoidStarImport]
[WARN] model/Incident.java:6:1: Missing a Javadoc comment. [MissingJavadocType]
... 110 more warnings ...
Audit done.
Warnings: 116

# AFTER: fixed source (backend), run 2026-10-18T13:15:52+00:00
$ java -jar checkstyle-9.3-all.jar -c /google_checks.xml backend/src/main/java
Starting audit...
Audit done.
Warnings: 0

# AFTER: fixed source (backend-spring4-test), run 2026-10-18T13:15:59+00:00
$ java -jar checkstyle-9.3-all.jar -c /google_checks.xml backend-spring4-test/src/main/java
Starting audit...
Audit done.
Warnings: 0
```

Figure 2.1: Checkstyle 9.3 — 116 warnings on the original source, 0 on the fixed backend modules (18 Oct 2026)

SpotBugs summary (archived XML)

```
# Archived report from the 16 Oct 2026 run (before the fixes): evidence/archived-2026-10-16-spring4-run/spotbugsXml.xml
SpotBugs version      : 4.9.8
Analysed              : /app/target/classes (backend-spring4-test)
Packages / classes    : 7 / 32
Java                  : 17.0.15
Timestamp             : Fri, 16 Oct 2026 17:15:28 +0000
BugInstance elements  : 0
total_bugs            : 0
```

Figure 2.2: SpotBugs summary — 0 bugs (archived report, build of 16 Oct 2026)

Frontend: npm ci · vite build · npm audit · 18 Oct 2026

```
# Run: 2026-10-18T13:16:09+00:00  node v22.22.0  npm 10.9.4
$ npm ci

added 70 packages, and audited 71 packages in 3s

found 0 vulnerabilities
$ npm run build

> enterprise-service-desk@1.0.0 build
> vite build

vite v7.3.7 building client environment for production...
transforming...
✓ 27 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                  0.37 kB | gzip:  0.27 kB
dist/assets/index-DMN28Om0.css  236.25 kB | gzip: 32.47 kB
dist/assets/index-nJQ8AcXa.js   158.24 kB | gzip: 50.26 kB
✓ built in 1.31s
$ npm audit
found 0 vulnerabilities
```

Figure 2.3: Frontend clean install, production build and npm audit — 0 vulnerabilities (18 Oct 2026)

SCREENSHOT TO INSERT — Figure 2.4

SpotBugs and Trivy results on the final source

Run scripts\verify-all.ps1, then screenshot backend-spring4-test-03-spotbugs.log (BUILD SUCCESS) and trivy-servicedesk-backend-sb4_local.log

Figure 2.4: SpotBugs and Trivy results on the final source

TESTING EXECUTION

- **Test Framework Used:** JUnit 5 (Jupiter) with Spring Boot Test and Spring MockMvc, run by Maven Surefire against an in-memory H2 database in MySQL mode. An end-to-end smoke test (Node.js 22, scripts/smoke-test.mjs) runs against the deployed Docker stack and MySQL.
- **Test Cases Table:** the eight automated test methods in the source, with the assertion each one makes.

Test ID	Test Level (Unit / API)	Function / Endpoint Tested	Input / Action	Expected Result	Status (Pass/ Fail)
TC-01	Unit	IncidentMetrics.awaitingAction()	OPEN, ASSIGNED and RESOLVED incidents	Returns 2 (only OPEN/ASSIGNED counted)	**Pass***
TC-02	Unit	IncidentMetrics.completed()	OPEN, RESOLVED and CLOSED incidents	Returns 2 (RESOLVED/CLOSED counted)	**Pass***
TC-03	API	POST /api/auth/register	Email "invalid"	Status 400 Bad Request	**Pass***
TC-04	API	POST /api/incidents	Valid token and incident body	Status 201, status = OPEN	**Pass***
TC-05	API	POST /api/incidents	Blank title	Status 400 Bad Request	**Pass***
TC-06	API	GET /api/incidents	No token	Status 403 Forbidden	**Pass***
TC-07	API	GET /api/reports/summary	End-user token	Status 403 Forbidden	**Pass***
TC-08	API	GET /api/incidents	Valid end-user token	Status 200, JSON array	**Pass***

***** *Status from the Maven Surefire reports of the 16 Oct 2026 run of the Spring Boot 4 module (8 run, 0 failures, 0 errors; Figure 3.1). The 18 Oct changes did not alter any assertion. Each API test first registers a new user (POST /api/auth/register, expects 201). Test method names are listed in docs/TEST_CASES_VERIFIED_FROM_SOURCE.md.*

3.1 End-to-End Smoke Test

Login, assignment, status changes and notifications are not covered by separate unit tests, so scripts/smoke-test.mjs checks them against the running stack with real MySQL. It runs 20 checks: valid and invalid login (200/401), request without a token, incident creation (201, OPEN) and blank-title validation (400), role-scoped lists, admin assignment to an engineer, engineer progress with a comment, resolution refused without resolution text (400) and accepted with it, audit history (CREATED, ASSIGNED, STATUS_CHANGED, COMMENT, RESOLUTION), assignment and resolution notifications, the administrator dashboard counts, and denials for an employee updating a ticket, an employee opening the report, an unassigned engineer viewing a ticket and an engineer changing roles. verify-all runs it on the Spring Boot 3 backend and again after switching to the Spring Boot 4 backend on the same database, and checks that the schema is unchanged.

Execution Proof:

JUnit 5 / Surefire results (archived XML)

```
# Archived Surefire XML from the 16 Oct 2026 run (Spring Boot 4.1.1, Java 17.0.15, WSL2)

com.bea.servicedesk.IncidentMetricsTest  tests=2 failures=0 errors=0 skipped=0 time=0.132s
  PASS  completedCountsResolvedAndClosed            0.114s
  PASS  awaitingActionCountsOnlyOpenAndAssigned     0.004s

com.bea.servicedesk.ApiIntegrationTest  tests=6 failures=0 errors=0 skipped=0 time=86.565s
  PASS  unauthenticatedListRejected                14.293s
  PASS  incidentRequiresTitle                       1.221s
  PASS  incidentCreated                             1.064s
  PASS  invalidEmailRejected                        0.220s
  PASS  userCannotAccessAdminReport                 0.303s
  PASS  userCanListOwnIncidents                     0.499s

Total: 8 tests, 0 failures, 0 errors, 0 skipped
```

Figure 3.1: JUnit 5 results from the Surefire reports — 8 tests, 0 failures, 0 errors (run of 16 Oct 2026)

SCREENSHOT TO INSERT — Figure 3.2
Maven test run on the final source: "Tests run: 8, Failures: 0, Errors: 0 — BUILD SUCCESS"
From evidence\runs\<timestamp>\backend-01-mvn-test.log and backend-spring4-test-01-mvn-test.log

Figure 3.2: Maven test run on the final source: "Tests run: 8, Failures: 0, Errors: 0 — BUILD SUCCESS"

SCREENSHOT TO INSERT — Figure 3.3
End-to-end smoke test: "20/20 checks passed"
From evidence\runs\<timestamp>\stack-b-sb4-smoke-test.log (and stack-a-sb3-smoke-test.log)

Figure 3.3: End-to-end smoke test: "20/20 checks passed"

3.2 Evidence Status

Evidence	Status	Where
8 JUnit tests	Passed on 16 Oct 2026 (archived XML)	evidence/archived-2026-10-16-spring4-run/
Checkstyle 0 warnings	Run on the final source, 18 Oct 2026	evidence/fresh-2026-10-18-this-environment/
npm build + audit	Run on the final source, 18 Oct 2026	evidence/fresh-2026-10-18-this-environment/
Tests, SpotBugs, Trivy, smoke test on final source	To run: scripts/verify-all	evidence/runs/<timestamp>/summary.md

CORE IMPLEMENTATION

Summary of the working features developed:

- **Module 1 (User Interface / Presentation):** a React single-page application with role-aware navigation for employees, support engineers and administrators. Screens: sign-in and self-registration; an overview dashboard with total, awaiting-action, in-progress and resolved counts and the latest incidents; a searchable incident list; a new-incident form (title, description, category, priority); a ticket detail panel with the activity timeline and an update form (assign engineer, status, resolution, comment); notifications with an unread badge; user-role administration; and status and priority reporting. API errors are shown using the "detail" message returned by the backend.

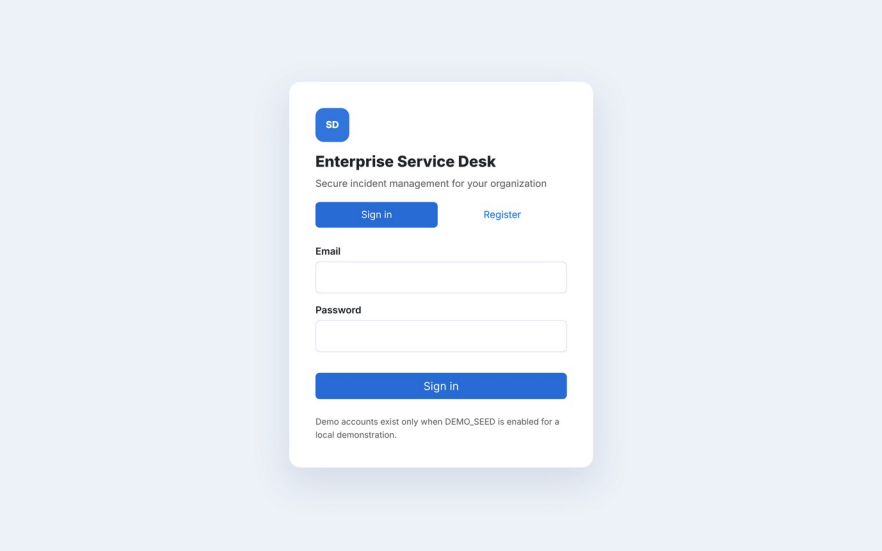


Figure 4.1: Sign-in screen (production build, 18 Oct 2026)

SCREENSHOT TO INSERT — Figure 4.2
Overview dashboard (administrator)
Run verify-all.ps1 -KeepRunning, open http://localhost:8089, sign in as admin@demo.local

Figure 4.2: Overview dashboard (administrator)

SCREENSHOT TO INSERT — Figure 4.3
Incident list with search (administrator or employee)
Incidents tab

Figure 4.3: Incident list with search (administrator or employee)

SCREENSHOT TO INSERT — Figure 4.4
New incident creation form (employee)
New incident tab, signed in as employee@demo.local

Figure 4.4: New incident creation form (employee)

SCREENSHOT TO INSERT — Figure 4.5
Ticket detail with activity timeline and update form (engineer)
Click a ticket, signed in as engineer@demo.local

Figure 4.5: Ticket detail with activity timeline and update form (engineer)

- **Module 2 (Core Business Logic / Processing):** incident workflow management in IncidentService and AccessService. An employee submits an incident; the request is validated and saved as OPEN with a CREATED audit event. An administrator assigns it to a user with the ENGINEER role (status

ASSIGNED, assignment event, notification to the engineer). The assigned engineer moves it to IN_PROGRESS or ESCALATED, adds comments and records a resolution, then RESOLVED or CLOSED; each change notifies the requester. Rules enforced on the server:

- Employees see only their own incidents, engineers only those assigned to them, administrators all.
- Only employees and administrators create incidents; only engineers and administrators update them; only administrators assign.
- A ticket cannot be set back to OPEN or ASSIGNED, and RESOLVED/CLOSED require resolution text.
- Administrators manage roles but cannot change their own role; reports are administrator-only.
- Each change and its audit events and notifications are saved in one transaction (@Transactional).

- **Module 3 (Data Persistence & Integration):** Spring Data JPA repositories map four entities to MySQL 8.4 tables and use derived queries for role-scoped lists (for example findByAssigneeIdOrderByCreatedAtDesc). Integration is REST/JSON: the browser calls /api on Nginx, which forwards to Spring Boot; every request carries a Bearer JWT that the security filter verifies. No external email or SMS API is connected.

Table	Key columns	Purpose
users	id, name, email (unique), password_hash, role	Accounts with BCrypt hashes and one role
incidents	id, title, description, category, priority, status, requester_id, assignee_id, resolution, created_at, updated_at	Tickets
incident_events	id, incident_id, actor_id, action, detail, created_at	Audit timeline
notifications	id, recipient_id, message, read_flag, created_at	In-app notifications

Schema safety: Hibernate runs with JPA_DDL_AUTO=validate by default, so the application never alters an existing schema; update is used only once to create a new local database, and the Spring Boot 4 override forces validate. scripts/backup-db takes a mysqldump before any upgrade. Secrets come only from environment variables, the backend refuses a JWT secret shorter than 32 bytes, and demo accounts exist only when DEMO_SEED=true with a DEMO_PASSWORD supplied at run time.

SCREENSHOT TO INSERT — Figure 4.6
Database rows written by the workflow (incidents and incident_events)
From evidence\runs\<timestamp>\stack-b-sb4-db-rows.log

Figure 4.6: Database rows written by the workflow (incidents and incident_events)

4.1 Implementation Boundaries

Not implemented from the Phase-1 target architecture: email/SMS delivery, file attachments, category management, automatic SLA escalation, database replication, monitoring and cloud deployment. Versioned database migrations (Flyway) are the next planned step for production use.

EVALUATION RUBRICS

Construction Readiness, Testing and Implementation

(60 MARKS Scaled Down to 10 marks)

S.No	Evaluation Criteria	Excellent	Good	Needs Improvement	Max Marks	Marks Obtained
1.	Repo Setup & Code Structure	Well-organized GitHub/GitLab repo setup with modular folder organization. **(9-10 marks)**	Fairly organized repository with minor structural flaws. **(6-8 marks)**	Disorganized repo; lacks clear modular structure. **(0-5 marks)**	10	
2.	Static Code Analysis	Tool executed; zero critical bugs; report submitted. **(13-15 marks)**	Tool executed; minor code bugs present. **(9-12 marks)**	Static analysis not performed or results ignored. **(0-8 marks)**	15	
3.	Testing Execution	Working Unit and API/Integration tests with passing logs **(17-20 marks)**	Working unit tests, but limited integration test coverage. **(11-16 marks)**	Missing unit/integration tests or execution errors. **(0-10 marks)**	20	
4.	Implementation	Functional implementation of core modules matching architectural design. **(13-15 marks)**	Partial implementation with minor functional bugs or gaps **(9-12 marks)**	Incomplete or non-functional module implementation. **(0-8 marks)**	15	
				TOTAL MARKS		

Signature of Course Co-Ordinator

REFERENCES/BIBLIOGRAPHY

[1] Spring, “Spring Boot Reference Documentation,” versions 3.5 and 4.1. [Online]. Available: https://docs.spring.io/spring-boot/

[2] Spring, “Spring Security Reference: Servlet Applications,” [Online]. Available: https://docs.spring.io/spring-security/reference/

[3] Spring, “Spring Data JPA Reference Documentation,” [Online]. Available: https://docs.spring.io/spring-data/jpa/reference/

[4] Hibernate, “Hibernate ORM User Guide,” [Online]. Available: https://hibernate.org/orm/documentation/

[5] Oracle, “MySQL 8.4 Reference Manual,” [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/

[6] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, May 2015.

[7] M. Nottingham, E. Wilde and S. Dalal, “Problem Details for HTTP APIs,” IETF RFC 9457, Jul. 2023.

[8] Meta Open Source, “React Documentation,” [Online]. Available: https://react.dev/

[9] Bootstrap Team, “Bootstrap 5.3 Documentation,” [Online]. Available: https://getbootstrap.com/docs/5.3/

[10] JUnit Team, “JUnit 5 User Guide,” [Online]. Available: https://junit.org/junit5/docs/current/user-guide/

[11] SpotBugs Project, “SpotBugs Manual,” version 4.9. [Online]. Available: https://spotbugs.readthedocs.io/

[12] Checkstyle Project, “Checkstyle — Google Java Style Checks,” version 9.3. [Online]. Available: https://checkstyle.org/

[13] Google, “Google Java Style Guide,” [Online]. Available: https://google.github.io/styleguide/javaguide.html

[14] Aqua Security, “Trivy Documentation,” [Online]. Available: https://trivy.dev/

[15] Docker Inc., “Docker Compose Documentation,” [Online]. Available: https://docs.docker.com/compose/

[16] OWASP Foundation, “Application Security Verification Standard 4.0.3,” 2021. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/

All online sources accessed on 18 Oct. 2026.